

2026 春《操作系统》课程实验报告

实验 1

23301037 客昱阳

一.准备工作

- 将分支切换到实验一所需的 `experiment1` 分支

```
kyy008@Kyy008deMacBook-Pro:~/Desktop/Kyy008/File/school/3th_2/OS/ex...
~/De/K/F/sc/3th_2/0/experiment main
git checkout experiment1
Switched to branch 'experiment1'
Your branch is up to date with 'origin/experiment1'.
~/De/K/F/sc/3th_2/0/experiment experiment1
```

- 检查一下上次环境配置的 docker 容器已经启动

```
~/De/K/F/sc/3th_2/0/experiment experiment1
docker ps -a --filter name=gardeneros

CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS
PORTS         NAMES
7b99f7bbef25   openeuler/openeuler   "/bin/bash"             11 days ago   Up 4 minutes
gardeneros
```

- 进入容器

```
~/De/K/F/sc/3th_2/0/experiment experiment1 !1 ?1
docker exec -it gardeneros bash

Welcome to 6.19.13-orbstack-gbd1dc07b8cf4
```

- 检查版本

```
[root@7b99f7bbef25 mnt]# rustc --version
cargo --version
rustc 1.97.0-nightly (ca9a134e0 2026-04-26)
cargo 1.97.0-nightly (eb9b60f1f 2026-04-24)
[root@7b99f7bbef25 mnt]#
```

二.实验步骤

1.创建 Rust 项目

- 创建一个新的可执行程序项目

Bash

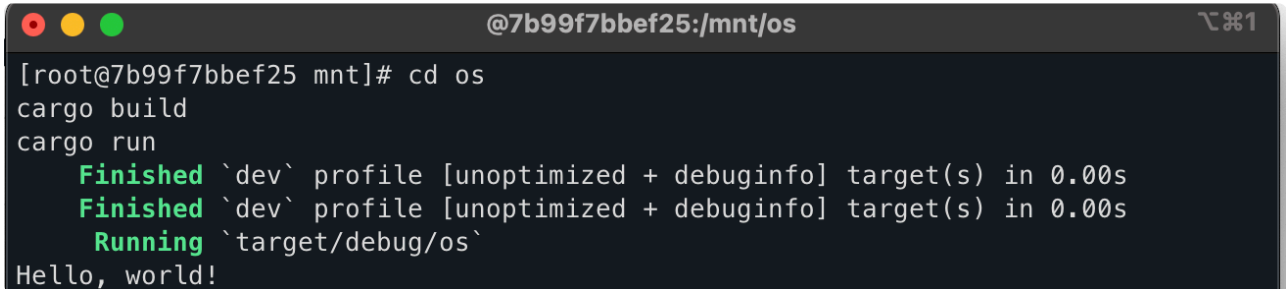
```
1 cargo new os --bin
```

```
[root@7b99f7bbef25 mnt]# cargo new os --bin
    Creating binary (application) `os` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
```

- 编译运行默认 Rust 程序

Bash

```
1 cd os
2 cargo build
3 cargo run
```



```
@7b99f7bbef25:/mnt/os
[root@7b99f7bbef25 mnt]# cd os
cargo build
cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/os`
Hello, world!
```

2.移除标准库依赖

- 修改 target 为 riscv64，默认把这个项目编译成 RISC-V 裸机程序

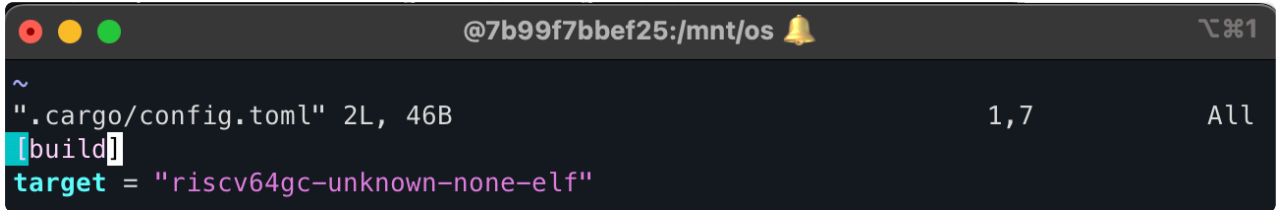
Bash

```
1 mkdir .cargo
2 vim .cargo/config.toml
```

输入下方内容

Bash

```
1 [build]
2 target = "riscv64gc-unknown-none-elf"
```



```
~
".cargo/config.toml" 2L, 46B 1,7 All
[build]
target = "riscv64gc-unknown-none-elf"
```

- 修改 main.rs 文件

不使用 Rust 标准库 std 和默认的 main 入口，并实现 panic handler

Bash

```
1 vim src/main.rs
```

修改为下面内容

Bash

```
1 #![no_std]
2 #![no_main]
3
4 use core::panic::PanicInfo;
5
6 #[panic_handler]
7 fn panic(_info: &PanicInfo) -> ! {
8     loop {}
9 }
```

```
@7b99f7bbef25:/mnt/os
~
~
"src/main.rs" 3L, 45B      1,1      All
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}
```

- 编译这个 no_std 空程序

```
Bash

1 cargo build
```

```
@7b99f7bbef25:/mnt/os
[root@7b99f7bbef25 os]# cargo build
  Compiling os v0.1.0 (/mnt/os)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.21s
[root@7b99f7bbef25 os]#
```

- 提交 git

```
Bash

1 git add .
2 git commit -m "feat(exp1): build no_std riscv executable skeleton - 233
   01037 客昱阳"
3 git push
```

```

~/De/K/F/sc/3th_2/0/experiment  experiment1 !2 ?2
git add .

~/De/K/F/sc/3th_2/0/experiment  experiment1 +7
git commit -m "feat(exp1): build no_std riscv executable skeleton - 23301037 客昱阳"
[experiment1 ad1e89f] feat(exp1): build no_std riscv executable skeleton - 23301037 客昱阳
7 files changed, 213 insertions(+), 118 deletions(-)
delete mode 100644 env.txt
create mode 100644 os/.cargo/config.toml
create mode 100644 os/Cargo.lock
create mode 100644 os/Cargo.toml
create mode 100644 os/src/main.rs
create mode 100644 "\345\256\236\351\252\214\357\274\232\347\213\254\347\253\213\347\232\204\345\217\257\346\211\247\350\241\214\347\250\213\345\272\217.txt"

~/De/K/F/sc/3th_2/0/experiment  experiment1 :1
git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (11/11), 3.36 KiB | 3.36 MiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Kyy008/OS-experiment
80a05b0..ad1e89f  experiment1 -> experiment1

```

• 分析独立的可执行程序

Bash

- 1 file target/riscv64gc-unknown-none-elf/debug/os
- 2 rust-readobj -h target/riscv64gc-unknown-none-elf/debug/os
- 3 rust-objdump -S target/riscv64gc-unknown-none-elf/debug/os

```

@7b99f7bbef25:/mnt/os
[root@7b99f7bbef25 os]# file target/riscv64gc-unknown-none-elf/debug/os
target/riscv64gc-unknown-none-elf/debug/os: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked, with debug_info, not stripped
[root@7b99f7bbef25 os]#

```

```

@7b99f7bbef25:/mnt/os
lly linked, with debug_info, not stripped
[root@7b99f7bbef25 os]# rust-readobj -h target/riscv64gc-unknown-none-elf/debug/os

File: target/riscv64gc-unknown-none-elf/debug/os
Format: elf64-littleriscv
Arch: riscv64
AddressSize: 64bit
LoadName: <Not found>
ElfHeader {
  Ident {
    Magic: (7F 45 4C 46)
    Class: 64-bit (0x2)
    DataEncoding: LittleEndian (0x1)
    FileVersion: 1
    OS/ABI: SystemV (0x0)
    ABIVersion: 0
    Unused: (00 00 00 00 00 00 00)
  }
  Type: Executable (0x2)
  Machine: EM_RISCV (0xF3)
  Version: 1
  Entry: 0x0
  ProgramHeaderOffset: 0x40
  SectionHeaderOffset: 0xE90
  Flags [ (0x5)
    EF_RISCV_FLOAT_ABI_DOUBLE (0x4)
    EF_RISCV_RVC (0x1)
  ]
  HeaderSize: 64
  ProgramHeaderEntrySize: 56
  ProgramHeaderCount: 4
  SectionHeaderEntrySize: 64
  SectionHeaderCount: 12
  StringTableSectionIndex: 10
}
[root@7b99f7bbef25 os]# rust-objdump -S target/riscv64gc-unknown-none-elf/debug/os

target/riscv64gc-unknown-none-elf/debug/os:      file format elf64-littleriscv
[root@7b99f7bbef25 os]#

```

可以发现：

`file`：生成的是 64-bit RISC-V ELF 可执行文件

`rust-readobj`：入口地址 Entry 是 0x0

`rust-objdump`：没有反汇编代码

印证了现在是一个空程序，因为还没有入口函数 `_start`。

3. 用户态可执行的环境

- 增加入口函数 `_start`

Bash

```
1 vim src/main.rs
```

新增 `_start` 入口

注意，由于本实验使用 2026 版本 Rust，新增 `_start` 入口时使用 `#[unsafe(no_mangle)]`。

其原因是 `no_mangle` 会影响最终生成的符号名，在新版 Rust 中属于 unsafe attribute，需要显式标记。

```
@7b99f7bbef25:/mnt/os
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

#[unsafe(no_mangle)]
extern "C" fn _start() -> ! {
    loop {}
}
```

- 重新编译运行

Bash

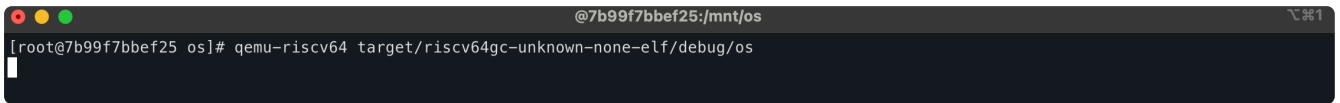
```
1 cargo build
```

```
[root@7b99f7bbef25 os]# cargo build
  Compiling os v0.1.0 (/mnt/os)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.36s
[root@7b99f7bbef25 os]#
```

运行程序

Bash

```
1 qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os
```



```
@7b99f7bbef25:/mnt/os
[root@7b99f7bbef25 os]# qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os
```

发现程序一直运行不结束，这是因为 `_start` 里是： `loop {}`

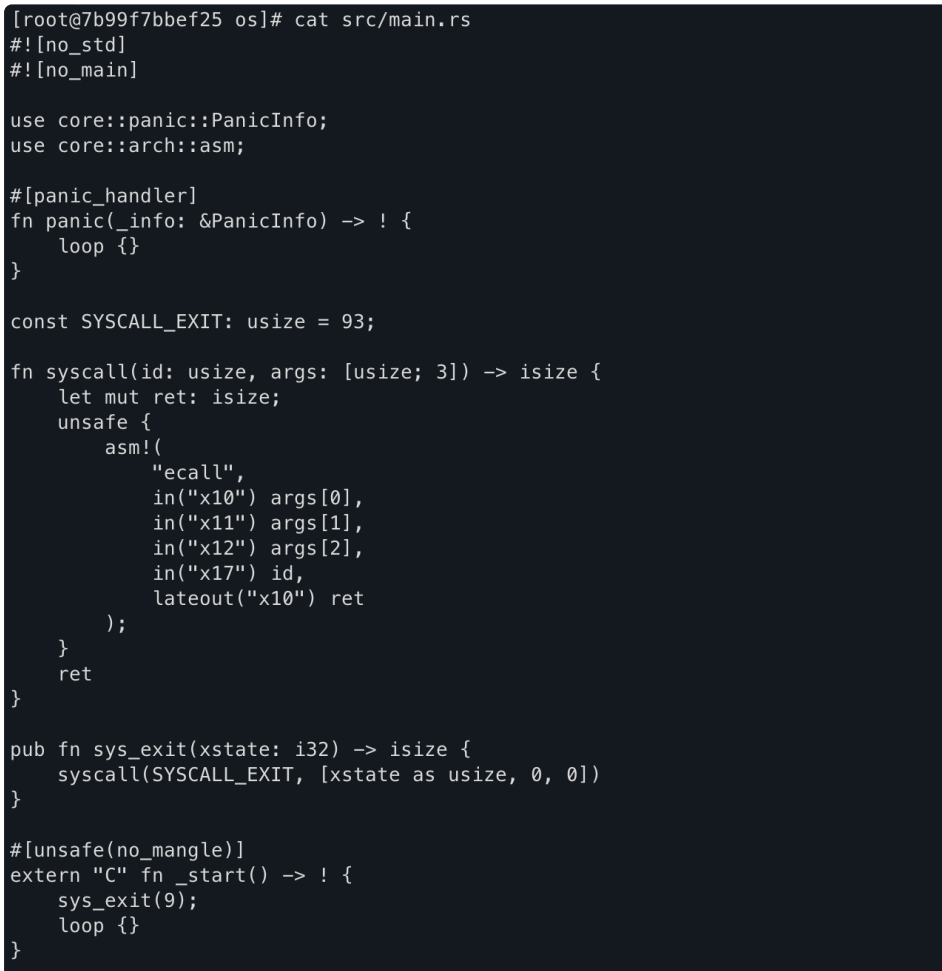
- 实现退出机制

用 Linux 系统调用 `exit` 正常退出程序。

Bash

```
1 vim src/main.rs
```

将文件修改为如下所示



```
[root@7b99f7bbef25 os]# cat src/main.rs
#![no_std]
#![no_main]

use core::panic::PanicInfo;
use core::arch::asm;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

const SYSCALL_EXIT: usize = 93;

fn syscall(id: usize, args: [usize; 3]) -> isize {
    let mut ret: isize;
    unsafe {
        asm!(
            "ecall",
            in("x10") args[0],
            in("x11") args[1],
            in("x12") args[2],
            in("x17") id,
            lateout("x10") ret
        );
    }
    ret
}

pub fn sys_exit(xstate: i32) -> isize {
    syscall(SYSCALL_EXIT, [xstate as usize, 0, 0])
}

#[unsafe(no_mangle)]
extern "C" fn _start() -> ! {
    sys_exit(9);
    loop {}
}
```

重新编译运行验证退出机制

Bash

```
1 cargo build
2 qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os
```

```
[root@7b99f7bbef25 os]# cargo build
qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os
  Compiling os v0.1.0 (/mnt/os)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.25s
```

- 实现输出支持 `sys_write`

现在程序能正常退出了，但还不能打印内容。

接下来封装 Linux 的 `write` 系统调用

Bash

```
1 vim src/main.rs
```

将文件修改为如下所示


```
@7b99f7bbef25:/mnt/os
#![no_main]

use core::panic::PanicInfo;
use core::arch::asm;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

const SYSCALL_EXIT: usize = 93;
const SYSCALL_WRITE: usize = 64;

fn syscall(id: usize, args: [usize; 3]) -> isize {
    let mut ret: isize;
    unsafe {
        asm!(
            "ecall",
            in("x10") args[0],
            in("x11") args[1],
            in("x12") args[2],
            in("x17") id,
            lateout("x10") ret
        );
    }
    ret
}

pub fn sys_exit(xstate: i32) -> isize {
    syscall(SYSCALL_EXIT, [xstate as usize, 0, 0])
}

pub fn sys_write(fd: usize, buffer: &[u8]) -> isize {
    syscall(SYSCALL_WRITE, [fd, buffer.as_ptr() as usize, buffer.len()])
}

#[unsafe(no_mangle)]
extern "C" fn _start() -> ! {
    sys_exit(9);
    loop {}
}

-- INSERT --
```

实现 print 函数。 `sys_write` 只能写字节，现在封装成 Rust 的格式化输出。

Bash

```
1 vim src/main.rs
```

将文件修改为如下所示

```

@7b99f7bbef25:/mnt/os
use core::fmt::{self, Write};
use core::panic::PanicInfo;
use core::arch::asm;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

const SYSCALL_EXIT: usize = 93;
const SYSCALL_WRITE: usize = 64;

fn syscall(id: usize, args: [usize; 3]) -> isize {
    let mut ret: isize;
    unsafe {
        asm!(
            "ecall",
            in("x10") args[0],
            in("x11") args[1],
            in("x12") args[2],
            in("x17") id,
            lateout("x10") ret
        );
    }
    ret
}

pub fn sys_exit(xstate: i32) -> isize {
    syscall(SYSCALL_EXIT, [xstate as usize, 0, 0])
}

pub fn sys_write(fd: usize, buffer: &[u8]) -> isize {
    syscall(SYSCALL_WRITE, [fd, buffer.as_ptr() as usize, buffer.len()])
}

struct Stdout;

impl Write for Stdout {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        sys_write(1, s.as_bytes());
        Ok(())
    }
}

pub fn print(args: fmt::Arguments) {
    Stdout.write_fmt(args).unwrap();
}

```

有了 `print` 函数后，再封装成平时常用的宏

Bash

```
1 vim src/main.rs
```

在 `print` 函数下追加下图所示代码

```

}
}

pub fn print(args: fmt::Arguments) {
    Stdout.write_fmt(args).unwrap();
}

#[macro_export]
macro_rules! print {
    ($($arg:tt)*) => {
        $crate::print(format_args!($($arg)*));
    };
}

#[macro_export]
macro_rules! println {
    () => {
        $crate::print!("\n");
    };
    ($($arg:tt)*) => {
        $crate::print!("{}", format_args!($($arg)*));
    };
}

```

start 函数改为下图所示代码

```

#[unsafe(no_mangle)]
extern "C" fn _start() -> ! {
    println!("Hello, world!");
    sys_exit(9);
    loop {}
}

-- INSERT --
72,31 Bot

```

重新编译运行后，期望输出 `Hello, world!`

Bash

```

1 cargo build
2 qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os

```

```

@7b99f7bbef25:/mnt/os
[root@7b99f7bbef25 os]# vim src/main.rs
[root@7b99f7bbef25 os]# cargo build
qemu-riscv64 target/riscv64gc-unknown-none-elf/debug/os
  Compiling os v0.1.0 (/mnt/os)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.09s
Hello, world!
[root@7b99f7bbef25 os]#

```

三.问题回答

1.为什么称最后实现的程序为独立的可执行程序，它和标准的程序有什么区别？

因为它不依赖 Rust 标准库，也不使用 Rust 默认的 `main` 入口，而是通过自己定义 `_start` 作为程序入口。同时，程序中的退出和输出也不是通过标准库完成的，而是直接使用系统调用实现。

它和标准程序的区别在于：标准 Rust 程序通常依赖 std 标准库，由运行时环境负责初始化并调用 `main` 函数，也可以直接使用 `println!` 等标准库功能；而本实验中的独立可执行程序需要自己处理入口函数、panic 处理、系统调用、退出和输出等内容，更接近操作系统底层程序的运行方式。

2. 实现和编译独立可执行程序的目的是什么？

理解程序在没有标准库和普通运行时支持的情况下如何运行。通过手动实现 `_start`、`sys_exit`、`sys_write` 和输出宏，可以理解可执行程序的入口、退出机制以及系统调用的基本过程。

同时，这也是操作系统实验的基础。后续编写操作系统内核或裸机程序时，运行环境往往没有完整的标准库支持，因此需要掌握如何构建一个最小可运行程序，并在底层环境中完成基本的输出和退出操作。